

数据结构核心知识点详细讲解

第一章 绪论

基本概念和术语

- **数据**：计算机可处理的各种符号集合，包括数值、字符、图像等。
- **数据元素**：数据的基本单位，是计算机处理的独立个体，如链表中的一个结点、数组中的一个元素。
- **数据项**：数据元素的最小不可分割单位，如学生记录中的“学号”“姓名”。
- **数据结构**：相互之间存在特定关系的数据元素的集合，包含逻辑结构、存储结构和数据运算三要素。
 - 逻辑结构：数据元素之间的逻辑关系，分为线性结构（如线性表、栈、队列）和非线性结构（如树、图）。
 - 存储结构：数据在计算机中的存储方式，主要有顺序存储（如数组）、链式存储（如链表）、索引存储和散列存储。
 - 数据运算：对数据结构中的元素进行的操作，如插入、删除、查找、排序等。
- **算法**：解决特定问题的有限指令序列，具有有穷性、确定性、可行性、输入和输出五大特性。

程序时间复杂度计算

- **定义**：衡量算法执行时间随问题规模增长的变化趋势，只关注增长最快的项，忽略常数项和低次项。
- **计算步骤**：
 1. 找出算法中执行次数最多的核心操作（基本操作）。
 2. 计算核心操作的执行次数与问题规模 n 的函数关系 $T(n)$ 。
 3. 对 $T(n)$ 取渐进复杂度，即保留最高次项并去掉系数，得到时间复杂度 $O(f(n))$ 。
- **常见时间复杂度（从低到高）**：
 - $O(1)$ ：常数阶，如直接访问数组元素。
 - $O(\log n)$ ：对数阶，如二分查找。
 - $O(n)$ ：线性阶，如遍历数组。
 - $O(n \log n)$ ：线性对数阶，如归并排序、快速排序。
 - $O(n^2)$ ：平方阶，如双重循环的冒泡排序。
 - $O(n^3)$ ：立方阶，如三重循环的矩阵乘法。
 - $O(2^n)$ ：指数阶，如递归求解汉诺塔问题。
- **示例**：

```
for(int i=0; i<n; i++){
    for(int j=0; j<i; j++){
        printf("hello"); // 核心操作
    }
}
```

核心操作执行次数为 $1+2+\dots+(n-1)=n(n-1)/2$ ，时间复杂度为 $O(n^2)$ 。

第二章 线性表

线性表的基本概念

- **定义：** n ($n \geq 0$) 个数据元素的有限序列，记作 $(a_1, a_2, \dots, a_n)$ ，其中 n 为表长度， $n=0$ 时为空表。
- **特点：**
 1. 元素具有相同特性（数据类型一致）。
 2. 元素间存在序偶关系，除首元素外每个元素有唯一直接前驱，除尾元素外每个元素有唯一直接后继。

顺序表的基本操作

- **定义：** 用连续的存储空间（数组）存储线性表元素，元素的物理位置与逻辑顺序一致。
- **存储结构：**

```
#define ListSize 100 // 最大长度
typedef int ListData;
typedef struct {
    ListData *data; // 存储空间基址
    int length; // 当前元素个数
} SeqList;
```

- **核心操作：**
 1. **初始化：** 分配存储空间，初始化长度为0。

```
void InitList(SeqList &L) {
    L.data = (ListData *)malloc(ListSize * sizeof(ListData));
    if (!L.data) exit(1); // 分配失败
    L.length = 0;
}
```

2. **插入：** 在第 i 个位置插入元素 x ，需移动后续元素。

```
int Insert(SeqList &L, ListData x, int i) {
    if (i < 0 || i > L.length || L.length == ListSize) return 0;
    for (int j = L.length; j > i; j--) {
        L.data[j] = L.data[j-1]; // 后移元素
    }
    L.data[i] = x;
    L.length++;
    return 1;
}
```

3. **删除：** 删除第 i 个元素，需移动后续元素填补空缺。

```
int Delete(SeqList &L, int i) {
    if (i < 0 || i >= L.length) return 0;
    for (int j = i; j < L.length-1; j++) {
        L.data[j] = L.data[j+1]; // 前移元素
    }
    L.length--;
    return 1;
}
```

4. **查找**：按值查找元素x，返回其位置（未找到返回-1）。

```
int Find(SeqList &L, ListData x) {
    for (int i = 0; i < L.length; i++) {
        if (L.data[i] == x) return i;
    }
    return -1;
}
```

单链表的基本操作

- **定义**：用任意存储单元存储元素，每个元素（结点）包含数据域和指针域，指针域指向后继结点。
- **存储结构**：

```
typedef char ListData;
typedef struct node {
    ListData data; // 数据域
    struct node *link; // 指针域（指向后继结点）
} ListNode;
typedef ListNode *LinkList; // 头指针
```

- **核心操作**：

1. **初始化**：头指针置空。

```
void InitList(LinkList &first) {
    first = NULL;
}
```

2. **插入**：

- 表头插入：新结点指针指向原头结点，头指针更新为新结点。
- 中间/表尾插入：找到第i-1个结点，新结点指针指向其后续结点，第i-1个结点指针指向新结点。

```
int Insert(LinkList &first, ListData x, int i) {
    ListNode *p = first;
    int k = 0;
    while (p != NULL && k < i-1) { // 找第i-1个结点
        p = p->link;
    }
    // 创建新结点并插入
    // ...
}
```

```

        k++;
    }
    if (p == NULL && i != 1) return 0; // 位置无效
    ListNode *newnode = (ListNode *)malloc(sizeof(ListNode));
    newnode->data = x;
    if (i == 1 || first == NULL) { // 表头插入
        newnode->link = first;
        first = newnode;
    } else { // 中间/表尾插入
        newnode->link = p->link;
        p->link = newnode;
    }
    return 1;
}

```

3. **删除**: 找到第*i*-1个结点, 跳过第*i*个结点(修改指针), 释放第*i*个结点空间。

```

int Delete(LinkList &first, int i) {
    ListNode *p = first, *q;
    if (i == 1) { // 删除表头
        q = first;
        first = first->link;
    } else {
        int k = 0;
        while (p != NULL && k < i-1) { // 找第i-1个结点
            p = p->link;
            k++;
        }
        if (p == NULL || p->link == NULL) return 0; // 位置无效
        q = p->link;
        p->link = q->link; // 跳过待删除结点
    }
    free(q);
    return 1;
}

```

4. **查找**: 按值查找, 返回结点指针(未找到返回NULL)。

```

ListNode *Find(LinkList first, ListData x) {
    ListNode *p = first;
    while (p != NULL && p->data != x) {
        p = p->link;
    }
    return p;
}

```

顺序表和链表的应用：链表合并

- **问题**: 将两个有序单链表(递增)合并为一个有序单链表。
- **算法思路**:
 1. 初始化合并后链表的头指针和尾指针。
 2. 同时遍历两个链表, 比较当前结点值, 将较小值结点接入合并链表。

3. 遍历结束后，将剩余结点接入合并链表。

- 代码实现：

```
LinkList MergeList(LinkList A, LinkList B) {
    LinkList C = NULL, rear = NULL;
    while (A != NULL && B != NULL) {
        if (A->data <= B->data) {
            if (C == NULL) {
                C = A;
                rear = A;
            } else {
                rear->link = A;
                rear = A;
            }
            A = A->link;
        } else {
            if (C == NULL) {
                C = B;
                rear = B;
            } else {
                rear->link = B;
                rear = B;
            }
            B = B->link;
        }
    }
    // 接入剩余结点
    if (A != NULL) rear->link = A;
    if (B != NULL) rear->link = B;
    return C;
}
```

循环链表操作

- **定义：**表尾结点的指针域指向头结点（或表头结点），形成闭合循环，分为单循环链表和双向循环链表。

- **单循环链表特点：**

1. 从任意结点可遍历整个链表。
2. 空表时头指针指向自身（带表头结点）。

- **核心操作（以带表头结点为例）：**

1. **初始化：**

```
void InitList(LinkList &first) {
    first = (ListNode *)malloc(sizeof(ListNode));
    first->link = first; // 表尾指向自身
}
```

2. **插入：**与单链表类似，表尾插入时新结点指针指向头结点。

3. **删除：**删除表尾结点时，需找到表尾前驱结点，使其指针指向头结点。

双向循环链表操作

- **定义：**每个结点包含前驱指针（prior）、数据域（data）和后继指针（next），表尾结点next指向头结点，头结点prior指向表尾结点。
- **存储结构：**

```
typedef int ListData;
typedef struct dnode {
    ListData data;
    struct dnode *prior, *next;
} DbListNode;
typedef DbListNode *DbList;
```

- **核心操作：**

1. 初始化：

```
void InitList(DbList &first) {
    first = (DbListNode *)malloc(sizeof(DbListNode));
    first->prior = first;
    first->next = first; // 循环指向自身
}
```

2. 插入：在结点p后插入新结点q。

```
int Insert(DbList first, DbListNode *p, ListData x) {
    DbListNode *q = (DbListNode *)malloc(sizeof(DbListNode));
    q->data = x;
    q->prior = p;
    q->next = p->next;
    p->next->prior = q;
    p->next = q;
    return 1;
}
```

3. 删除：删除结点p。

```
int Delete(DbList first, DbListNode *p) {
    if (p == first) return 0; // 不能删除表头
    p->prior->next = p->next;
    p->next->prior = p->prior;
    free(p);
    return 1;
}
```

第三章 栈和队列

栈的定义与特点

- **定义：**限定仅在表尾（栈顶top）进行插入（进栈）和删除（出栈）操作的线性表，表首为栈底（bottom）。
- **特点：**后进先出（LIFO），即最后进栈的元素最先出栈。
- **抽象数据类型：**

```
template <class Type>
class Stack {
public:
    void Push(Type &item); // 进栈
    Type Pop(); // 出栈
    Type GetTop(); // 取栈顶元素
    bool IsEmpty(); // 判断栈空
    bool IsFull(); // 判断栈满
};
```

栈的存储结构与基本操作

1. 顺序栈（数组实现）

- **存储结构：**

```
#define STACK_INIT_SIZE 100
#define STACKINCREMENT 10
typedef int SElemType;
typedef struct {
    SElemType *base; // 栈底指针
    SElemType *top; // 栈顶指针（指向栈顶元素下一个位置）
    int stacksize; // 当前栈容量
} SqStack;
```

- **核心操作：**

1. 初始化：

```
void InitStack(SqStack &S) {
    S.base = (SElemType *)malloc(STACK_INIT_SIZE * sizeof(SElemType));
    if (!S.base) exit(1);
    S.top = S.base;
    S.stacksize = STACK_INIT_SIZE;
}
```

2. 进栈：

```
void Push(SqStack &S, SElemType e) {
    if (S.top - S.base >= S.stacksize) { // 栈满, 扩容
        S.base = (SElemType *)realloc(S.base, (S.stacksize +
STACKINCREMENT) * sizeof(SElemType));
        S.top = S.base + S.stacksize;
        S.stacksize += STACKINCREMENT;
    }
    *S.top++ = e; // 元素入栈, 栈顶指针上移
}
```

3. 出栈:

```
int Pop(SqStack &S, SElemType &e) {
    if (S.top == S.base) return 0; // 栈空
    e = *--S.top; // 栈顶指针下移, 取出元素
    return 1;
}
```

4. 取栈顶元素:

```
int GetTop(SqStack S, SElemType &e) {
    if (S.top == S.base) return 0;
    e = *(S.top - 1);
    return 1;
}
```

2. 链式栈 (链表实现)

- 存储结构:

```
typedef int ElemType;
typedef struct LNode {
    ElemType data;
    struct LNode *next;
} LNode;
typedef struct {
    LNode *top; // 栈顶指针 (链头)
    int length; // 元素个数
} LStack;
```

- 核心操作:

1. 初始化:

```
void InitStack(LStack &S) {
    S.top = NULL;
    S.length = 0;
}
```


2. 进栈:

```
void Push(LStack &S, ElemType e) {
    LNode *p = (LNode *)malloc(sizeof(LNode));
    p->data = e;
    p->next = S.top; // 新结点指向原栈顶
    S.top = p; // 栈顶指针更新
    S.length++;
}
```

3. 出栈:

```
int Pop(LStack &S, ElemType &e) {
    if (S.top == NULL) return 0;
    LNode *p = S.top;
    e = p->data;
    S.top = p->next; // 栈顶指针下移
    free(p);
    S.length--;
    return 1;
}
```

表达式求值（栈的应用）

- 问题：计算中缀表达式（如3*(7-2)）的值，需先转换为后缀表达式（逆波兰式），再通过栈求值。
- 核心思路：
 1. 中缀转后缀：利用栈处理运算符，按优先级出栈入栈，操作数直接输出。
 - 运算符优先级：括号>乘除>加减。
 - 规则：遇到操作数输出；遇到左括号入栈；遇到右括号弹出栈中运算符直到左括号；遇到运算符，弹出栈中优先级≥当前运算符的运算符，再入栈；遍历结束后弹出剩余运算符。
 2. 后缀求值：利用栈存储操作数，遇到运算符弹出两个操作数计算，结果入栈，最终栈顶为结果。
- 示例：中缀表达式“3*(7-2)”求值全过程
 1. 中缀转后缀：
 - 输入：3*(7-2)#
 - 后缀输出：3 7 2 - *
 2. 后缀求值：

步骤	输入字符	栈操作	栈内容
1	3	进栈	[3]
2	7	进栈	[3,7]
3	2	进栈	[3,7,2]

步骤	输入字符	栈操作	栈内容
4	-	7、2出栈，计算7-2=5，5进栈	[3,5]
5	*	3、5出栈，计算3*5=15，15进栈	[15]
6	#	结束，栈顶为结果15	[15]

队列的定义与特点

- **定义：**限定在表尾（队尾rear）插入、表头（队头front）删除的线性表。
- **特点：**先进先出（FIFO），即最先入队的元素最先出队。
- **抽象数据类型：**

```
template <class Type>
class Queue {
public:
    void EnQueue(Type &item); // 入队
    Type DeQueue(); // 出队
    Type GetFront(); // 取队头元素
    bool IsEmpty(); // 判断队空
    bool IsFull(); // 判断队满
};
```

队列的存储结构

1. 链队列（链表实现）

- **存储结构：**

```
typedef int QElemType;
typedef struct QNode {
    QElemType data;
    struct QNode *next;
} QNode, *QueuePtr;
typedef struct {
    QueuePtr front; // 队头指针（指向头结点）
    QueuePtr rear; // 队尾指针（指向队尾结点）
} LinkQueue;
```

- **核心操作：**

1. 初始化：

```
void InitQueue(LinkQueue &Q) {
    Q.front = Q.rear = (QueuePtr)malloc(sizeof(QNode));
    Q.front->next = NULL;
}
```

2. 入队：

```
void EnQueue(LinkQueue &Q, QElemType e) {
    QueuePtr p = (QueuePtr)malloc(sizeof(QNode));
    p->data = e;
    p->next = NULL;
    Q.rear->next = p;
    Q.rear = p; // 队尾指针更新
}
```

3. 出队:

```
int DeQueue(LinkQueue &Q, QElemType &e) {
    if (Q.front == Q.rear) return 0; // 队空
    QueuePtr p = Q.front->next;
    e = p->data;
    Q.front->next = p->next;
    if (Q.rear == p) Q.rear = Q.front; // 队中仅一个元素
    free(p);
    return 1;
}
```

2. 循环队列 (数组实现)

- 存储结构:

```
#define MAXSIZE 100
typedef int QElemType;
typedef struct {
    QElemType *base; // 存储空间基址
    int front; // 队头指针
    int rear; // 队尾指针 (指向队尾元素下一个位置)
} SqQueue;
```

- 核心特性:

1. 队空条件: $\text{front} == \text{rear}$ 。
2. 队满条件: $(\text{rear} + 1) \% \text{MAXSIZE} == \text{front}$ (牺牲一个位置避免与队空混淆)。
3. 入队: $\text{rear} = (\text{rear} + 1) \% \text{MAXSIZE}$ 。
4. 出队: $\text{front} = (\text{front} + 1) \% \text{MAXSIZE}$ 。

- 核心操作:

1. 初始化:

```
void InitQueue(SqQueue &Q) {
    Q.base = (QElemType *)malloc(MAXSIZE * sizeof(QElemType));
    Q.front = Q.rear = 0;
}
```

2. 入队:

```
int EnQueue(SqQueue &Q, QElemType e) {
    if ((Q.rear + 1) % MAXSIZE == Q.front) return 0; // 队满
    Q.base[Q.rear] = e;
    Q.rear = (Q.rear + 1) % MAXSIZE;
    return 1;
}
```

3. 出队:

```
int DeQueue(SqQueue &Q, QElemType &e) {
    if (Q.front == Q.rear) return 0; // 队空
    e = Q.base[Q.front];
    Q.front = (Q.front + 1) % MAXSIZE;
    return 1;
}
```

第六章 树和二叉树

二叉树的定义

- **定义:** n ($n \geq 0$) 个结点的有限集合, 空树 ($n=0$) 或由一个根结点和两棵互不相交的左子树、右子树组成, 左子树和右子树均为二叉树。
- **特点:** 每个结点至多有两棵子树 (度 ≤ 2), 子树有左右之分 (有序树)。
- **五种基本形态:** 空树、仅根结点、根+左子树、根+右子树、根+左子树+右子树。

二叉树的性质证明

性质1: 第 i 层至多有 2^{i-1} 个结点 ($i \geq 1$)

- **证明** (数学归纳法):
 1. 基例: $i=1$ 时, 只有根结点, $2^{1-1}=1$, 成立。
 2. 归纳假设: 假设第 j 层 ($j < i$) 至多有 2^{j-1} 个结点。
 3. 归纳递推: 第 i 层结点由第 $i-1$ 层结点的子结点构成, 每个结点至多有2个子结点, 故第 i 层至多有 $2 \times 2^{i-2} = 2^{i-1}$ 个结点, 成立。

性质2: 深度为 k 的二叉树至多有 $2^k - 1$ 个结点 ($k \geq 1$)

- **证明:**

深度为 k 的二叉树, 每层结点数取最大值 (性质1), 总结点数为 $2^0 + 2^1 + \dots + 2^{k-1} = 2^k - 1$ (等比数列求和)。

性质3: 对任意二叉树, 叶结点数 n_0 =度为2的结点数 n_2+1

- **证明:**
 1. 设总结点数为 n , 度为1的结点数为 n_1 , 则 $n = n_0 + n_1 + n_2$ 。
 2. 总边数 e : 每个度为1的结点贡献1条边, 度为2的结点贡献2条边, 叶结点贡献0条边, 故 $e = n_1 + 2n_2$ 。

3. 二叉树中边数 $e = n - 1$ (除根结点外每个结点有一条父边), 故 $n_1 + 2n_2 = n_0 + n_1 + n_2 - 1$, 化简得 $n_0 = n_2 + 1$ 。

性质4: 具有n个结点的完全二叉树深度为 $\lfloor \log_2 n \rfloor + 1$

- 证明:

1. 设深度为 h , 由性质2, 深度为 $h-1$ 的二叉树至多有 $2^{h-1} - 1$ 个结点, 深度为 h 的至多有 $2^h - 1$ 个结点。
2. 完全二叉树深度为 h , 故 $2^{h-1} - 1 < n \leq 2^h - 1 \rightarrow 2^{h-1} \leq n < 2^h$ 。
3. 取对数得 $h-1 \leq \log_2 n < h$, 因 h 为整数, 故 $h = \lfloor \log_2 n \rfloor + 1$ 。

二叉树的构建

1. 基于二叉链表的存储结构

```
typedef char TreeData;
typedef struct node {
    TreeData data;
    struct node *leftChild, *rightChild; // 左、右子树指针
} BinTreeNode;
typedef BinTreeNode *BinTree;
```

2. 按前序序列构建二叉树 (递归)

- 思路: 前序序列顺序为“根→左→右”, 用特殊字符 (如 '@') 表示空结点。
- 示例: 前序序列“A B @ @ D E @ G @ @ F @ @”对应二叉树:



- 代码实现:

```
void CreateBiTree(BinTree &T) {
    char ch;
    scanf("%c", &ch);
    if (ch == '@') T = NULL; // 空结点
    else {
        T = (BinTreeNode *)malloc(sizeof(BinTreeNode));
        T->data = ch;
        CreateBiTree(T->leftChild); // 构建左子树
        CreateBiTree(T->rightChild); // 构建右子树
    }
}
```

3. 基于层次序列构建完全二叉树

- **思路：**按层次顺序（自上而下、自左向右）输入结点值，利用完全二叉树的父子结点索引关系（父 $i \rightarrow$ 左子 $2i+1$ 、右子 $2i+2$ ）构建。
- **代码实现：**

```
void CreateCompleteBiTree(BinTree &T, TreeData arr[], int n, int i) {
    if (i >= n) {
        T = NULL;
        return;
    }
    T = (BinTreeNode *)malloc(sizeof(BinTreeNode));
    T->data = arr[i];
    CreateCompleteBiTree(T->leftChild, arr, n, 2*i+1); // 左子树
    CreateCompleteBiTree(T->rightChild, arr, n, 2*i+2); // 右子树
}
// 调用: TreeData arr[] = {'A','B','C','D','E'}; CreateCompleteBiTree(T, arr, 5, 0);
```

第七章 图

图的定义

图是由顶点集合（vertex）和顶点间关系集合（边或弧）组成的数据结构，记作 $\text{Graph} = (V, E)$ 。

- **顶点集合 V：**有穷非空的顶点（数据元素）集合。
- **边/弧集合 E：**顶点间关系的有穷集合。
 - 无向图：E 中元素为无序对 (u, v) ，称 u 与 v 互为邻接顶点，边无方向。
 - 有向图：E 中元素为有序对 $\langle x, y \rangle$ （称为弧）， x 为弧尾（起点）， y 为弧头（终点），弧有方向。
- **核心相关概念：**
 - 完全图：n 个顶点的无向完全图有 $n(n-1)/2$ 条边，有向完全图有 $n(n-1)$ 条边。
 - 权与网络：边带相关数值（权）的图称为网络，权可表示距离、代价等。
 - 顶点的度：无向图中顶点 v 的度是关联边的条数（ $TD(v)$ ）；有向图中分为入度（ $ID(v)$ ，以 v 为弧头的弧数）和出度（ $OD(v)$ ，以 v 为弧尾的弧数），度 = 入度 + 出度。
 - 路径与回路：顶点序列 $(v_i, v_{p1}, \dots, v_j)$ 为从 v_i 到 v_j 的路径，路径长度为边数（非带权图）或边权之和（带权图）；起点与终点重合的路径为回路（环）。
 - 连通性：无向图中任意两顶点有路径则为连通图，非连通图的极大连通子图为连通分量；有向图中任意两顶点互有路径则为强连通图，其极大强连通子图为强连通分量。

图的存储结构

1. 邻接表 (Adjacency List)

邻接表是图的链式存储结构，适合存储稀疏图，核心是“顶点表 + 边表”的组合。

- **结构组成：**
 - 顶点表：每个顶点对应一个结点，包含顶点数据和指向第一条依附该顶点的边/弧的指针。
 - 边表（弧表）：每个边/弧对应一个结点，包含目标顶点下标、边权（带权图）和指向下一条边/弧的指针。
- **存储结构定义：**

```
#define MAX_VERTEX_NUM 20 // 最大顶点数
typedef char VertexType; // 顶点数据类型
typedef int EdgeData; // 边权类型（非带权图可省略或设为1）

// 边表结点（存储弧/边信息）
typedef struct ArcNode {
    int adjvex; // 目标顶点在顶点表中的下标
    EdgeData cost; // 边权（带权图用，非带权图可删除）
    struct ArcNode *nextarc; // 指向下一条弧/边的指针
} ArcNode;

// 顶点表结点（存储顶点信息及边表头指针）
typedef struct VNode {
    VertexType data; // 顶点数据
    ArcNode *firstarc; // 指向第一条依附该顶点的弧/边
} VNode, AdjList[MAX_VERTEX_NUM];

// 图的邻接表结构
typedef struct {
    AdjList vertices; // 顶点表数组
    int vexnum, arcnum; // 当前顶点数和边/弧数
    int kind; // 图的类型（0：无向图，1：有向图）
} ALGraph;
```

- **无向图与有向图的邻接表差异：**
 - 无向图：每条边对应两个边表结点（u 指向 v 和 v 指向 u），边表结点总数为 $2 \times \text{弧数}$ 。
 - 有向图：每条弧仅对应一个边表结点（弧尾指向弧头），边表结点总数等于弧数。

2. 逆邻接表 (Inverse Adjacency List)

逆邻接表是有向图的特殊邻接表，专门存储“入弧”信息：

- 顶点表中每个顶点的边表，存储以该顶点为**弧头**的所有弧（即指向该顶点的前驱顶点）。
- 用途：快速查询有向图中某个顶点的入度、所有前驱顶点，与邻接表（查出处、出度）互补。
- 结构示例：
 - 有向图顶点 A 有弧 $\langle B, A \rangle$ 和 $\langle C, A \rangle$ ，则逆邻接表中 A 的边表结点指向 B 和 C 的下标。

邻接表构建算法（无向图为例）

算法思路

1. 输入顶点数和边数，初始化顶点表（存储顶点数据，边表头指针置空）。
2. 逐条输入边的信息（起点、终点、边权），为每条边创建两个边表结点（无向图双向关联）。
3. 将边表结点分别插入到起点和终点的边表头部（头插法，插入效率高）。

代码实现

```
// 构建无向图的邻接表
void CreateALGraph(ALGraph &G) {
    printf("请输入顶点数和边数（用空格分隔）：");
    scanf("%d %d", &G.vexnum, &G.arcnum);
    G.kind = 0; // 0 表示无向图

    // 1. 初始化顶点表
    for (int i = 0; i < G.vexnum; i++) {
        printf("请输入第 %d 个顶点的数据：", i + 1);
        getchar(); // 吸收换行符
        scanf("%c", &G.vertices[i].data);
        G.vertices[i].firstarc = NULL; // 边表头指针初始为空
    }

    // 2. 逐条插入边（无向图，每条边对应两个边表结点）
    for (int k = 0; k < G.arcnum; k++) {
        int tail, head; // 边的起点、终点（顶点表下标）
        EdgeData weight; // 边权
        printf("请输入第 %d 条边的起点下标、终点下标、边权（用空格分隔）：", k + 1);
        scanf("%d %d %d", &tail, &head, &weight);

        // ① 创建边表结点，插入到起点 tail 的边表头部
        ArcNode *p1 = (ArcNode *)malloc(sizeof(ArcNode));
        p1->adjvex = tail;
        p1->cost = weight;
        p1->nextarc = G.vertices[tail].firstarc;
        G.vertices[tail].firstarc = p1;

        // ② 创建边表结点，插入到终点 head 的边表头部（无向图双向）
        ArcNode *p2 = (ArcNode *)malloc(sizeof(ArcNode));
        p2->adjvex = head;
        p2->cost = weight;
        p2->nextarc = G.vertices[head].firstarc;
        G.vertices[head].firstarc = p2;
    }
}
```

- 若构建有向图，只需删除“插入终点边表结点”的步骤（p2 相关代码），仅保留弧尾指向弧头的边表结点。
- 若为非带权图，可省略 weight 参数，边表结点中 cost 字段可删除或默认设为 1。

图的遍历算法

图的遍历是从某顶点出发访遍所有顶点且仅访问一次的过程，需用辅助数组 `visited[]` 标记访问状态（0：未访问，1：已访问），避免重复访问。核心算法有深度优先搜索（DFS）和广度优先搜索（BFS）。

1. 深度优先搜索（DFS）

算法思想

- 从起始顶点 v 出发，优先访问 v 的一个未访问邻接顶点 w_1 ，再从 w_1 出发递归访问其未访问邻接顶点 w_2 ，以此类推。
- 当顶点 u 的所有邻接顶点均已访问，回溯到上一个未遍历完邻接顶点的顶点，继续搜索，直到连通图中所有顶点访问完毕。
- 非连通图需遍历所有顶点，对未访问顶点重复上述过程。

代码实现（基于邻接表）

```
// 辅助数组：标记顶点是否被访问（全局或作为参数传递）
int visited[MAX_VERTEX_NUM] = {0};

// 深度优先搜索核心函数：从顶点 v（下标）出发遍历
void DFS(ALGraph G, int v) {
    // 访问顶点 v，输出其数据
    printf("%c ", G.vertices[v].data);
    visited[v] = 1; // 标记为已访问

    // 遍历 v 的所有邻接顶点
    ArcNode *p = G.vertices[v].firstarc;
    while (p != NULL) {
        int w = p->adjvex; // 邻接顶点的下标
        if (!visited[w]) { // 若未访问，递归访问
            DFS(G, w);
        }
        p = p->nextarc; // 访问下一个邻接顶点
    }
}

// 图的深度优先遍历（处理非连通图）
void GraphDFS(ALGraph G) {
    // 初始化访问标记数组
    memset(visited, 0, sizeof(visited));
    printf("深度优先遍历结果：");
    // 遍历所有顶点，对未访问顶点执行 DFS
    for (int i = 0; i < G.vexnum; i++) {
        if (!visited[i]) {
            DFS(G, i);
        }
    }
    printf("\n");
}
```

2. 广度优先搜索 (BFS)

算法思想

- 从起始顶点 v 出发，先访问 v 的所有未访问邻接顶点 $w_1, w_2, \dots, w_k$ (第一层)。
- 再依次访问 $w_1, w_2, \dots, w_k$ 的所有未访问邻接顶点 (第二层)，逐层遍历，直到连通图中所有顶点访问完毕。
- 需用队列存储当前层顶点，保证“先进先出”的逐层访问顺序；非连通图处理方式同 DFS。

代码实现 (基于邻接表 + 队列)

```
// 队列结构 (简化实现, 用于存储顶点下标)
typedef struct Queue {
    int data[MAX_VERTEX_NUM];
    int front, rear; // 队头、队尾指针
} Queue;

// 初始化队列
void InitQueue(Queue &Q) {
    Q.front = Q.rear = 0;
}

// 入队
void EnQueue(Queue &Q, int x) {
    if ((Q.rear + 1) % MAX_VERTEX_NUM == Q.front) return; // 队满
    Q.data[Q.rear] = x;
    Q.rear = (Q.rear + 1) % MAX_VERTEX_NUM;
}

// 出队
int DeQueue(Queue &Q) {
    if (Q.front == Q.rear) return -1; // 队空
    int x = Q.data[Q.front];
    Q.front = (Q.front + 1) % MAX_VERTEX_NUM;
    return x;
}

// 判断队列是否为空
int QueueEmpty(Queue Q) {
    return Q.front == Q.rear;
}

// 广度优先搜索核心函数: 从顶点 v (下标) 出发遍历
void BFS(ALGraph G, int v) {
    Queue Q;
    InitQueue(Q);

    // 访问顶点 v, 标记并入队
    printf("%c ", G.vertices[v].data);
    visited[v] = 1;
    EnQueue(Q, v);

    while (!QueueEmpty(Q)) {
        int u = DeQueue(Q); // 出队当前顶点
```

```

        ArcNode *p = G.vertices[u].firstarc; // 遍历其邻接顶点
        while (p != NULL) {
            int w = p->adjvex;
            if (!visited[w]) { // 未访问则访问、标记、入队
                printf("%c ", G.vertices[w].data);
                visited[w] = 1;
                EnQueue(Q, w);
            }
            p = p->nextarc;
        }
    }
}

// 图的广度优先遍历（处理非连通图）
void GraphBFS(ALGraph G) {
    memset(visited, 0, sizeof(visited));
    printf("广度优先遍历结果: ");
    for (int i = 0; i < G.vexnum; i++) {
        if (!visited[i]) {
            BFS(G, i);
        }
    }
    printf("\n");
}

```

图的连通性问题

连通分量 (Connected Component)

- 无向图为非连通图时，从某顶点出发遍历仅能访问其所在的极大连通子图（连通分量）。
- 算法思路：遍历所有顶点，对未访问顶点执行 DFS/BFS，每个遍历结果即为一个连通分量。
- 非连通图的生成森林：所有连通分量的生成树集合。

重连通分量 (Biconnected Component)

- 关节点：删去顶点 v 及依附边后，图分裂为多个连通分量，则 v 为关节点。
- 重连通图：无关节点的连通图，任意两顶点至少有两条独立路径。
- 求解方法：通过深度优先生成树，根结点有两个以上子女则为关节点；其他结点 u 存在子女 w ， w 无法通过回边访问 u 的祖先，则 u 为关节点。

最小生成树 (Minimum Cost Spanning Tree)

核心准则

- 包含 n 个顶点和 $n-1$ 条边，无回路，边权总和最小。

普里姆 (Prim) 算法

算法思想

- 从起始顶点 u_0 出发，逐步将顶点加入生成树集合 U ，每次选择“一端在 U 、一端不在 U ”的权值最小边，将对应顶点加入 U ，直到 U 包含所有顶点。
- 辅助数组：
 - `lowcost[]`：存储 U 外顶点到 U 内的最小边权。
 - `nearvex[]`：存储 U 外顶点距离 U 内最近的顶点下标。

代码实现

- 时间复杂度： $O(n^2)$ ，适用于边稠密图。

活动网络

1. AOV 网络 (顶点表示活动)

- 定义：用顶点表示活动，有向边 $\langle v_i, v_j \rangle$ 表示活动 V_i 先于 V_j 执行，无回路。
- 拓扑排序：
 - 功能：检测回路，生成满足前驱后继关系的线性序列。
 - 算法步骤：
 1. 初始化入度数组 `count[]`，统计各顶点入度。
 2. 入度为 0 的顶点入栈。
 3. 出栈顶点并输出，删除其出边，对应顶点入度减 1，入度为 0 则入栈。
 4. 输出顶点数小于总顶点数则存在回路。

2. AOE 网络 (边表示活动)

- 定义：用边表示活动，顶点表示事件，边权为活动持续时间，源点为起始事件，汇点为结束事件。
- 关键路径：从源点到汇点的最长路径，决定工程最短完成时间。
- 核心参数：
 - 事件最早开始时间 $Ve[i]$ ：源点到 V_i 的最长路径长度。
 - 事件最迟开始时间 $VI[i]$ ：保证汇点按时完成的 V_i 最迟开始时间。
 - 活动最早开始时间 $e[k]$ ：活动 $\langle v_i, v_j \rangle$ 对应 $e[k] = Ve[i]$ 。
 - 活动最迟开始时间 $l[k]$ ：活动 $\langle v_i, v_j \rangle$ 对应 $l[k] = VI[j] - \text{边权}$ 。
 - 关键活动： $e[k] = l[k]$ 的活动，组成关键路径。

最短路径

1. Dijkstra 算法 (单源最短路径，边权非负)

算法思想

- 按路径长度递增次序生成最短路径，用辅助数组 `dist[]` 存储源点到各顶点的当前最短路径长度，`path[]` 存储路径前驱。

代码实现

```

void Dijkstra(ALGraph G, int v0) {
    int n = G.vexnum;
    int *dist = (int *)malloc(n * sizeof(int));
    int *path = (int *)malloc(n * sizeof(int));
    int *s = (int *)calloc(n, sizeof(int)); // 已确定最短路径的顶点集

    // 初始化
    for (int i = 0; i < n; i++) {
        dist[i] = MaxValue;
        path[i] = -1;
    }
    ArcNode *p = G.vertices[v0].firstarc;
    while (p != NULL) {
        dist[p->adjvex] = p->cost;
        path[p->adjvex] = v0;
        p = p->nextarc;
    }
    dist[v0] = 0;
    s[v0] = 1;

    // 循环 n-1 次，确定 n-1 条最短路径
    for (int i = 0; i < n-1; i++) {
        // 找当前最短路径顶点
        int min = MaxValue;
        int u = -1;
        for (int j = 0; j < n; j++) {
            if (!s[j] && dist[j] < min) {
                min = dist[j];
                u = j;
            }
        }
        if (u == -1) break; // 源点无法到达其他顶点
        s[u] = 1;

        // 更新最短路径
        p = G.vertices[u].firstarc;
        while (p != NULL) {
            int w = p->adjvex;
            if (!s[w] && dist[u] + p->cost < dist[w]) {
                dist[w] = dist[u] + p->cost;
                path[w] = u;
            }
            p = p->nextarc;
        }
    }

    // 输出结果
    printf("源点 %c 到各顶点的最短路径: \n", G.vertices[v0].data);
    for (int i = 0; i < n; i++) {
        if (dist[i] == MaxValue) {
            printf("到 %c: 无路径\n", G.vertices[i].data);
            continue;
        }
    }
}

```

```
    }  
    printf("到 %c: 长度=%d, 路径: ", G.vertices[i].data, dist[i]);  
    // 回溯路径  
    int tmp[100], cnt = 0;  
    for (int j = i; j != -1; j = path[j]) {  
        tmp[cnt++] = j;  
    }  
    for (int j = cnt-1; j >= 0; j--) {  
        printf("%c", G.vertices[tmp[j]].data);  
        if (j > 0) printf("->");  
    }  
    printf("\n");  
}  
  
free(dist); free(path); free(S);  
}
```